

Appendix E:

Table of Acceptance Criteria Reporting & Testing

Backlog Item	Acceptance Criteria
Sprint Review & Retrospective	- When a sprint review is held, A reviewed backlog list is produced showing Done / Not Done items for that sprint - Retrospective is held; at least 3 concrete improvement actions are documented.
Canvas Assignments	- When the assignment is delivered, the submission must include the required files with no missing component.
Testing	- When test is completed for a sprint, The results are noted and ready to be implemented for the next sprints. - Testing is performed on each core flow (Create/setup event, update fields, save/load) and has at least one test case each, with clear input and outputs. - If Errors scenario occurs, the tested scenario is observed and documented.
5 User Interviews	- When user interviews are done, 5 people are interviewed and logged. - A set of premade questions needs to be answered while also keeping them consistent. - Interviews are completed and written, each recommendation maps to a backlog change (new item, modified item or priority change).
Write and Finalize Reports for exam	- When the report is finalized and contains all required sections. - Report is checked and every reference resolves to an existing labeled item (no broken "figure x/ missing appendix") - The final report is submitted.
Make an Oral presentation	- Slides are prepared and checked. - The presentation fits the time limit - Working Demo/flow - Appropriate material is provided and presented.

Frontend: Technical Implementation

Backlog Item	Acceptance Criteria
Standardized frontend formatting with shared CSS styles.	- When any main page is opened, then headings/body, spacing, and primary button styling match the shared style definition. - If a new component is added, it is compiled without importing page specific CSS files for base styling.
Inspiration for post-registration dashboard and input for event format	- When documented, at least 3 dashboard layout references and 3 event format input patterns are captured with short note on what will be reused/avoided. - Reviewed by the team, then a single chosen direction is implemented.
Create post-registration user dashboard	- When they can navigate to the dashboard. - When they see a list of available events or empty state message if none. - If an event exists, the dashboard loads it. - Each event shows important information. - When a dashboard fails to load data, a visible error appears and is handled in the appropriate way.
Text field Issues in manual setup	- When the user writes text exceeding a defined max length, the input is prevented or handled. - When user attempts to continue with an empty required field, the UI blocks progression and highlights the missing field(s) - When the field loses focus or “continue” is clicked, the value is treated as empty and validation triggers.
FAQ “chatbot” widget	- The user clicks the FAQ button; the widget opens. - The user can close it. - When the user interacts or changes pages, the widget state resets according to the intended design.

Decide later button logic separated from continue button	- Both buttons exist. - When “Decide later” is clicked, the application saves the current step as undecided and navigates to the next step. - Both buttons are independent of each other.
Adjust scoring weights in setup based on selected scoring type	- When a user changes the scoring type, the only weights relevant to that scoring type are shown or changed. - Requires weights rules as provided by Peers. - The user can switch and modify different weights and rest or mapped dynamically.
Connect new AI backend to new frontend	- When the user sends a message, the frontend calls the configured AI endpoint and displays the assistant's Response in the chat UI - If the AI takes too long to respond a visible loading state appears and the input state flows, showing “thinking” - If the AI endpoint returns an error, the response is received with an appropriate error message, which is shown to the user and can retry without refreshing the page.
Connect New frontend to db	- When the page opens, the fields are populated with persisted data, for existing events. - If the user edits fields, the changes are saved.
Input checklist items	- A checklist component, when checklist items are provided. - Each item displays its respective label and the user-provided variable.
Edit button in summary list on AI setup page	- When the summary list is displayed and the user clicks “Edit” on an item, the UI navigates/auto-scrolls to the corresponding field and switches focus to the input provided at a given time. - The user edits are saved and returned to the AI chat.
Improved dashboard functionality (detail view)	- When an event shows on the dashboard and the user clicks “detail view”, the app navigates to a detail page/section showing the full event configuration. - When the user returns to the dashboard, then the previous dashboard state is preserved or reset consistently per defined behavior. - If the event ID is invalid, the error is handled in an appropriate way and does not crash.
Navbar (every page)	- Given any main route/page, when it loads, the navbar is visible with consistent items and styling across pages. - If clicked, the navigation occurs then the route changes without an error.

Generating a back button for two pages	- When the user clicks “back”, the app returns to the previous page/state as defined. - No previous navigation history exists; the app navigates to a safe default page, rather than nothing.
Add correct descriptions for the different fields in the ranking templates	- When the template is displayed, each field shows the correct description text (tooltip/help text/ label) from the template definition - When swapping to a different template, the description changes to match the selected template.
Ability to change scoring type In SetupAI and SetupManual (and dashboard) – decide later	- When scoring type is shown in the AI chat, Manual or dashboard detail, the user can change the types in one place, then all views reflect the new scoring type after save/reload - The UI clearly shows scoring type as undecided and blocks any scoring-type dependent fields until decided (or hide them), consistently across pages. - When opening the sector, it always opens in a default way, if not made.
Expanded template configuration in manual setup	- When the user selects a template, all template specific fields appear with defaults and validation. - If the user changes template options and saves, the correct one is saved.
AI setup fixes – Auto Scroll	- If the chatbot updates one or more sidebar fields, the update is applied, and the sidebar scrolls to the updated fields and focuses on it. - When multiple changes, the last one is scrolled to.
Sponsor image upload for public card showcases and event image (also in setupMethod.tsx)	- When the user selects an image file, then only allowed types are accepted and invalid get corresponding error handling - Max file size rule.
The green FAQ button is overlapping the privacy button	- The faq button remains a fixed size and does not block or overlap with anything
Day/night mode toggle	- When the user switches modes, the colors update. - Decide a default mode based on the user’s device’s preferences.
Athlete registration	- When a file upload button is added to the athlete tab in the dashboard edit page, accepting .csv and .xlsx files. - Option to add athletes manually.

	- A visual representation of current athletes (list or table).
Visualize the preview alongside the setup page (not summary page)	- When the setup is open, the user edits any setup field, then the preview filed updates around the same time. - For missing fields, it provides a clear placeholder state indicating what is missing.
Refine All setup fields/checklist items to match peers	- That the current setup fields match what is expected on the deliverables. - Works as intended with the AI
Icons	- When icons have been implemented in the AI Setup and Summary page and there is a clear visual difference between separate checklist or list items.

Backend: Technical Implementation

Backlog Item	Acceptance Criteria
Remove backend/requirement.txt from git	- When the old requirement.txt is deleted and replaced with the new and updated one
Set up a Database (DB)	- When DB is connected and initialized. - The app can create and connect without errors - When applying setup instruction, the required tables are created successfully. - When backend starts then it fails with a clear error message.
DB CRUD operations	- When create/read/update/delete (CRUD) endpoints are called with valid data, the database reflects the change, and the API returns the expected status code - When the different operations are called, they work as intended. - They are tested to make sure it works
Choose an AI agent to use and implement it	- Find different ai to use, - Based on findings, choose one. - Inform and come to an agreement with the group.
Set up system prompt	- When ai session is starts, the system prompt is included in the first request for that session - The system prompt is updated in configuration. When the backend restarts, the new sessions use the updated prompt without code changes. - Set it up to have a personality. - Clear limitations (it won't do any linear algebra). - It stays professional throughout the conversation

DB image handling	- When saved, the image record is created and linked to the correct event id. - Handles unsupported file types and empty payload - The handling is tested.
Merge Frontend to Backend	- When the application starts, both frontend and backend work together. - When the frontend request calls an API route the is reaches the backend without errors in the default dev setup - Having a clear readme so anyone can run the application.
Improve AI setup chat flow	- When the user sends a message, the backend returns a response that includes any updates. - The user sends invalid or inappropriate messages; the message gets handled in an appropriate way. - The chat flow is smooth and ai responds as expected.
Connect sideview on frontend to actual progress made with AI chat	- Updating and populating the appropriate fields in the chat summary from the database. - The correct information is displayed in the corresponding fields. - When no progress is made, it is kept empty or handled.
FK PK relationship between events and images tables has no constraints	- When the schema is updated, (like creating an image) creating a new record, the DB enforces that event_id references an existing event. - If an event is deleted, the related images are also deleted. - When looking into the DB schema, the foreign key constraint is present.
Bug: Changes to events should update and not create whole new instances	- When updating an event works. - There is no new event. - Tested the functionality
Update EventState to contain all Information from an Event	- When eventstate matches the information needed for an event. - Tested and works without breaking anything.
Handle Participant list: CSV/Excel	- When a CSV/XLSX is uploaded, the valid participant rows are inserted/updated in the database, and the API returns count. - If a file is missing or has missing columns, it is handled in an appropriate way. - The file invalid rows are handled.

Optimize API prompt to lower token usage	- When different models are analyzed (gemini.1.5 – 2.5 etc.) and found the most reasonable to cost efficient model. - Tuned and improved system prompt.
Implemented session id to maintain event creation	- When a new session starts, the backend returns a session id. - The request with that id retrieve the same event creation state - Handle invalid/expired session id. - When two different session ids are used, their states do not interfere with each other.
Implement abuse protection	- When a clear limitation on token usage has been implemented, along with a notice presented to users maliciously spending AI tokens.
Bug: Delete event does not work	- When given an existing event id and sending a delete request, the records of the corresponding event are deleted from the database. - The behavior should match the given PK ID and eventually the FK
The “send” button is disabled when waiting for AI answer, but we are still able to send messages.	- When the AI is given a request, the user cannot send another message, or it puts the new message in a queue.